



石河子大学信息科学与技术学院 《智能计算系统》课程实验报告

实验名称:	实验一
学生姓名:	卢瑶
学 号:	20211018341
学 院:	信息科学与技术学院
专业年级:	计科 2021 级 5 班

实验一 神经网络基础

一、实验目的：

- 1、掌握神经网络的结构
- 2、掌握神经网络的编程流程
- 3、编写程序实现常用的操作

二、实验要求：

- 1、用 Tensorflow 编程框架和 Python 语言实现源程序的编写；
- 2、源程序应书写规范，适当添加注释；
- 3、实验课对源程序进行编辑、调试、运行；实验结束后尽快完成实验报告，并按规定时间上交实验报告。

三、实验内容

编写程序实现鸢尾花数据集分类。

四、实验结果

4.1 实验代码

本次实验的目的是通过实现鸢尾花数据集分类，来深入理解神经网络的结构和编程流程。为此，我们将使用 TensorFlow 框架和 Python 语言编写源程序，实现数据的加载、处理、模型的构建、训练和测试，以及结果的可视化。实验过程中，我们将遵循编程规范，并适当添加注释以确保代码的可读性。

(1) 数据集导入

```
1. from sklearn import datasets
2. from pandas import DataFrame
3. import pandas as pd
4.
5. x_data = datasets.load_iris().data # .data 返回 iris 数据集所有输入特征
6. y_data = datasets.load_iris().target # .target 返回 iris 数据集所有标签
7. print("x_data from datasets: \n", x_data)
8. print("y_data from datasets: \n", y_data)
9.
10.x_data = DataFrame(x_data, columns=['花萼长度', '花萼宽度', '花瓣长度', '花瓣宽度']) # 为表格增加行索引（左侧）和列标签（上方）
11.pd.set_option('display.unicode.east_asian_width', True) # 设置列名对齐
12.print("x_data add index: \n", x_data)
13.
```

```
14.x_data['类别'] = y_data # 新加一列，列标签为‘类别’，数据为y_data
15.print("x_data add a column: \n", x_data)
```

在以上代码中，我们使用鸢尾花数据集作为基础来深入理解和掌握神经网络的结构与编程流程。首先，通过 `sklearn.datasets` 模块导入了鸢尾花数据集，这个数据集包括了 150 个样本，每个样本由 4 个特征组成：花萼长度、花萼宽度、花瓣长度和花瓣宽度，并且每个样本都标记了其属于三种鸢尾花中的哪一种。最后，打印出数据集的特征和标签，从而快速了解了数据集的结构和内容。

为了方便地操作和分析这些数据，我们使用 python 生态的 `pandas` 库，将特征数据转换成 `DataFrame` 格式，并为每个特征列命名，使数据更加直观。为了保证列名在控制台输出时能够整齐对齐，设置了 `pandas` 的显示选项。随后，在 `DataFrame` 中增加了一个名为“类别”的新列，将标签数据（即鸢尾花的种类）与每个样本的特征整合在了一起。通过这样的处理，不仅让数据集的结构更加清晰，也方便了后续的数据分析和模型训练准备工作。

（2）鸢尾花分类模型训练

由于鸢尾花的数据集较为简单，因此我们这里使用单层感知机来进行模型的训练，训练代码如下所示：

```
1. # -*- coding: UTF-8 -*-
2. # 利用鸢尾花数据集，实现前向传播、反向传播，可视化 loss 曲线
3.
4. # 导入所需模块
5. import tensorflow as tf
6. from sklearn import datasets
7. from matplotlib import pyplot as plt
8. import numpy as np
9.
10.# 导入数据，分别为输入特征和标签
11.x_data = datasets.load_iris().data
12.y_data = datasets.load_iris().target
13.
14.# 随机打乱数据（因为原始数据是顺序的，顺序不打乱会影响准确率）
15.# seed: 随机数种子，是一个整数，当设置之后，每次生成的随机数都一样（为方便
    教学，以保每位同学结果一致）
16.np.random.seed(116) # 使用相同的 seed，保证输入特征和标签一一对应
17.np.random.shuffle(x_data)
18.np.random.seed(116)
19.np.random.shuffle(y_data)
```

```

20.tf.random.set_seed(116)
21.
22.# 将打乱后的数据集分割为训练集和测试集，训练集为前120行，测试集为后30行
23.x_train = x_data[:-30]
24.y_train = y_data[:-30]
25.x_test = x_data[-30:]
26.y_test = y_data[-30:]
27.
28.# 转换x的数据类型，否则后面矩阵相乘时会因数据类型不一致报错
29.x_train = tf.cast(x_train, tf.float32)
30.x_test = tf.cast(x_test, tf.float32)
31.
32.# from_tensor_slices 函数使输入特征和标签值一一对应。（把数据集分批次，每个批次batch组数据）
33.train_db = tf.data.Dataset.from_tensor_slices((x_train, y_train)).batch(32)
34.test_db = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(32)
35.
36.# 生成神经网络的参数，4个输入特征故，输入层为4个输入节点；因为3分类，故输出层为3个神经元
37.# 用tf.Variable()标记参数可训练
38.# 使用seed使每次生成的随机数相同（方便教学，使大家结果都一致，在现实使用时不写seed）
39.w1 = tf.Variable(tf.random.truncated_normal([4, 3], stddev=0.1, seed=1))
40.b1 = tf.Variable(tf.random.truncated_normal([3], stddev=0.1, seed=1))
41.
42.lr = 0.1 # 学习率为0.1
43.train_loss_results = [] # 将每轮的loss记录在此列表中，为后续画loss曲线提供数据
44.test_acc = [] # 将每轮的acc记录在此列表中，为后续画acc曲线提供数据
45.epoch = 500 # 循环500轮
46.loss_all = 0 # 每轮分4个step，loss_all记录四个step生成的4个loss的和
47.
48.# 训练部分
49.for epoch in range(epoch): #数据集级别的循环，每个epoch循环一次数据集
50.     for step, (x_train, y_train) in enumerate(train_db): #batch级别的循环，每个step循环一个batch
51.         with tf.GradientTape() as tape: # with结构记录梯度信息
52.             y = tf.matmul(x_train, w1) + b1 # 神经网络乘加运算

```

```

53.         y = tf.nn.softmax(y) # 使输出 y 符合概率分布（此操作后与独热
    码同量级，可相减求 Loss）
54.         y_ = tf.one_hot(y_train, depth=3) # 将标签值转换为独热码
    格式，方便计算 Loss 和 accuracy
55.         loss = tf.reduce_mean(tf.square(y_ - y)) # 采用均方误差损
    失函数 mse = mean(sum(y-out)^2)
56.         loss_all += loss.numpy() # 将每个 step 计算出的 Loss 累加，
    为后续求 Loss 平均值提供数据，这样计算的 Loss 更准确
57.         # 计算 Loss 对各个参数的梯度
58.         grads = tape.gradient(loss, [w1, b1])
59.
60.         # 实现梯度更新 w1 = w1 - lr * w1_grad    b = b - lr * b_grad
61.         w1.assign_sub(lr * grads[0]) # 参数 w1 自更新
62.         b1.assign_sub(lr * grads[1]) # 参数 b 自更新
63.
64.         # 每个 epoch，打印 Loss 信息
65.         print("Epoch {}, loss: {}".format(epoch, loss_all/4))
66.         train_loss_results.append(loss_all / 4) # 将 4 个 step 的 Loss 求平
    均记录在此变量中
67.         loss_all = 0 # loss_all 归零，为记录下一个 epoch 的 Loss 做准备
68.
69.         # 测试部分
70.         # total_correct 为预测对的样本个数，total_number 为测试的总样本数，将
    这两个变量都初始化为 0
71.         total_correct, total_number = 0, 0
72.         for x_test, y_test in test_db:
73.             # 使用更新后的参数进行预测
74.             y = tf.matmul(x_test, w1) + b1
75.             y = tf.nn.softmax(y)
76.             pred = tf.argmax(y, axis=1) # 返回 y 中最大值的索引，即预测的分
    类
77.             # 将 pred 转换为 y_test 的数据类型
78.             pred = tf.cast(pred, dtype=y_test.dtype)
79.             # 若分类正确，则 correct=1，否则为 0，将 bool 型的结果转换为 int 型
80.             correct = tf.cast(tf.equal(pred, y_test), dtype=tf.int32)
81.             # 将每个 batch 的 correct 数加起来
82.             correct = tf.reduce_sum(correct)
83.             # 将所有 batch 中的 correct 数加起来
84.             total_correct += int(correct)
85.             # total_number 为测试的总样本数，也就是 x_test 的行数，shape[0] 返
    回变量的行数
86.             total_number += x_test.shape[0]
87.         # 总的准确率等于 total_correct/total_number
88.         acc = total_correct / total_number

```

```

89.     test_acc.append(acc)
90.     print("Test_acc:", acc)
91.     print("-----")
92.
93. # 绘制 Loss 曲线
94. plt.title('Loss Function Curve') # 图片标题
95. plt.xlabel('Epoch') # x 轴变量名称
96. plt.ylabel('Loss') # y 轴变量名称
97. plt.plot(train_loss_results, label="$Loss$") # 逐点画出
    train_loss_results 值并连线, 连线图标是 Loss
98. plt.legend() # 画出曲线图标
99. plt.show() # 画出图像
100.
101. # 绘制 Accuracy 曲线
102. plt.title('Acc Curve') # 图片标题
103. plt.xlabel('Epoch') # x 轴变量名称
104. plt.ylabel('Acc') # y 轴变量名称
105. plt.plot(test_acc, label="$Accuracy$") # 逐点画出 test_acc 值并
    连线, 连线图标是 Accuracy
106. plt.legend()
107. plt.show()

```

为了基于单层感知机进行鸢尾花的分类，我们首先导入了必要的库：TensorFlow 用于构建和训练模型，sklearn 用于获取鸢尾花数据集，matplotlib 用于可视化训练过程中的 loss 和准确率曲线，以及 numpy 用于数据处理。

我们从 sklearn 加载鸢尾花数据集，这个数据集包含 150 个样本，每个样本有 4 个特征。为了提高模型的泛化能力，我们通过 numpy 的随机种子和随机打乱函数，确保了数据的随机性，同时保证了特征和标签之间的一一对应。

接下来，我们将数据集分为训练集和测试集，其中训练集包含 120 个样本，测试集包含 30 个样本。我们还将特征数据的类型转换为 TensorFlow 能够处理的浮点数格式。

我们利用 TensorFlow 的 `from_tensor_slices` 函数和 `batch` 方法将训练集和测试集分批次处理，这有助于在训练过程中优化内存消耗并提高计算效率。在构建模型的过程中，我们定义了一个简单的线性模型，该模型包含一个输入层和一个输出层，其中输入层有 4 个节点，对应于数据集的 4 个特征，输出层有 3 个节点，对应于数据集的 3 个类别。我们使用了带有截断正态分布的随机初始化方法来初始化模型的权重和偏置。

设置学习率为 0.1 之后，我们开始训练模型。训练过程包括 500 个 epochs，每个 epoch 我们都会通过梯度下降算法来优化模型的权重和偏置，以减少均方误差损失函数的值。在每个 epoch 结束时，我们都记录了平均 loss，并在训练结束后通过 matplotlib 绘制了 loss 的变化曲线。

为了评估我们的模型，最后计算了模型在测试集上的准确率，并绘制了准确率的变化曲线。这两条曲线为我们提供了模型训练过程中性能的直观展示，并帮助我们理解模型的学习效果。

4.2 实验运行结果

(1) p43_datasets.load_iris.py 运行结果

p43_datasets.load_iris.py 运行结果如下图所示：

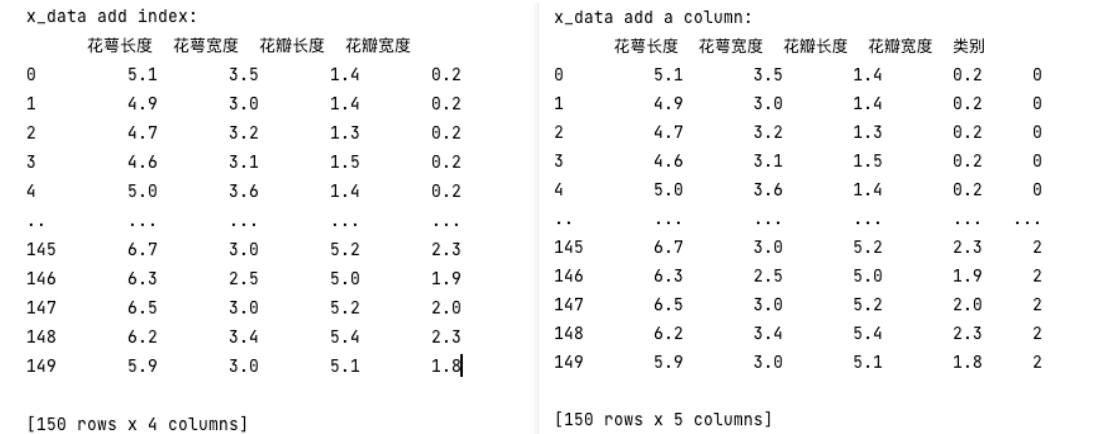


图 1 p43_datasets.load_iris.py 运行结果

由于篇幅有限，上图展示了最关键的两个结果截图。

通过精心设计的几步处理过程，我们对鸢尾花数据集进行了系统化的处理和展示，使得数据分析和后续的处理更加高效直观。首先，我们提取了每个样本的四个特征值：花萼长度、花萼宽度、花瓣长度和花瓣宽度，这些浮点数值精确反映了样本的实际测量结果。随后，我们呈现了样本的类别标签，这些标签以 0、1、2 的整数形式表示，分别对应鸢尾花的三个类别：Setosa、Versicolour 和 Virginica，为每个样本提供了明确的类别归属。

为了进一步提高数据的可操作性和易读性，我们将特征数据转换成了 Pandas DataFrame 格式，并为每个特征指定了直观的名称，同时确保了数据表格在输出时的整齐排版。此外，我们在 DataFrame 中新增了“类别”列，直接引用了原始的标签数据，这一举措不仅将每个样本的特征和相应的类别标签整合到

了同一个数据结构中，也使得每个样本的信息变得完整，便于进一步的数据探索和分析。

总的来说，我们的处理过程不仅限于数据的加载和初步展示，而是通过将数据转换为结构化、易于操作的格式，并整合了特征数据与类别标签，为数据集提供了完整的视图。

(2) p45_iris.py 运行结果

p45_iris.py 运行结果如下图所示：

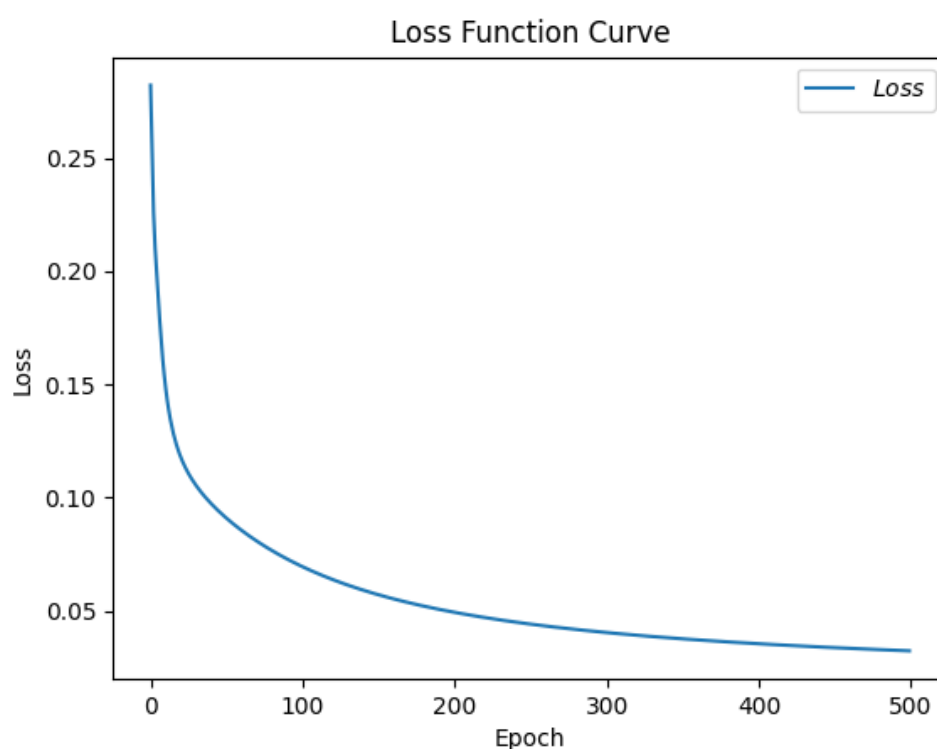


图 2 损失函数曲线

从上图中，我们可以看到：

1. **快速下降**：在最初的训练阶段，损失迅速下降，这通常表示模型在学习过程中迅速从随机初始化的权重调整到能够对训练数据进行有效预测的权重。
2. **逐渐趋于平稳**：随着 Epoch 数量的增加，损失下降的速度减慢，并开始趋于稳定。这意味着模型在逐步接近其性能的上限，进一步的学习改进变得较为缓慢。
3. **收敛情况**：在图的右侧，可以观察到曲线趋于平坦，这表示模型可能接近了

收敛，即进一步的训练不再导致损失的显著下降。

4. **无过拟合迹象**：由于损失函数曲线平滑且持续下降，没有上升的趋势或波动，这暗示模型没有出现过拟合的问题。如果出现过拟合，我们通常会看到损失值在减少到某一点后又开始增加。

综合来看，这个损失函数曲线表明了一个良好的训练过程，模型的学习是有效的，并且在所展示的训练周期内持续改进了其性能。

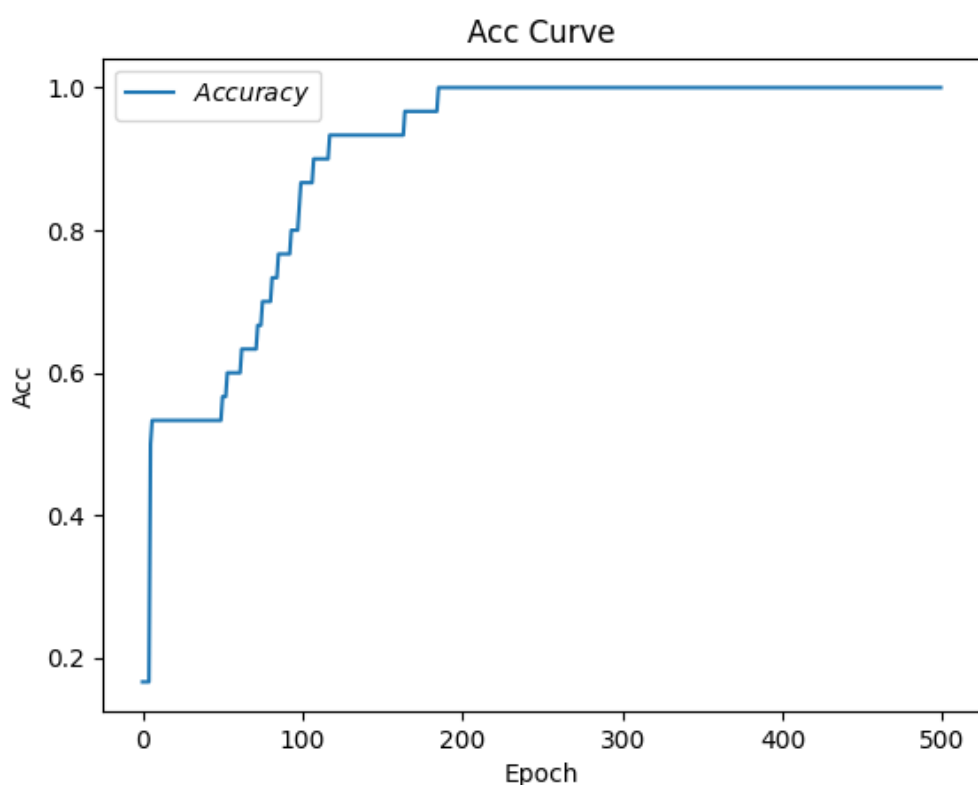


图 3 准确率变化趋势图

从上图不难看出：

1. **快速提升**：在训练的早期阶段，准确率快速提升，这通常意味着模型从初始状态开始迅速学习，有效地识别出数据中的模式和关系。
2. **逐渐稳定**：随着 Epoch 数增加，准确率的增长逐渐放缓，最终在某一点之后趋于平稳。这表明模型在逐步优化，但随着训练的进行，获得显著改进的机会减少。
3. **高准确率**：曲线在大约 80% 的 Epoch 数之后达到了高准确率（接近或等于 1.0），这表明模型在训练集上有着非常好的表现。

4. **潜在的过拟合**：由于准确率非常高，接近 100%，如果这个曲线代表的是训练集上的准确率，那么可能存在过拟合的风险。换句话说，模型可能太过于适应了训练数据，可能在未见过的测试数据上表现得不那么好。
5. **收敛表现**：准确率曲线在达到高值后变得非常平稳，这通常意味着模型已经达到了学习的极限，并且在给定的架构和数据集上进一步提升空间有限。

五、实验小结

5.1 数据集

鸢尾花数据集是由 150 个样本组成的多变量数据集，每个样本包含了四个特征：花萼长度、花萼宽度、花瓣长度和花瓣宽度，这些特征用于描述鸢尾花的形态特点。数据集中的样本被分为三个不同的类别，每个类别 50 个样本，分别代表鸢尾花的三个亚种：Setosa、Versicolour 和 Virginica。在处理过程中，我们将这些数据整合到一个结构化的表格中，使得每个样本的特征和标签都清晰地展现出来，为数据分析和模型建立提供方便。

5.2 鸢尾花训练程序流程图

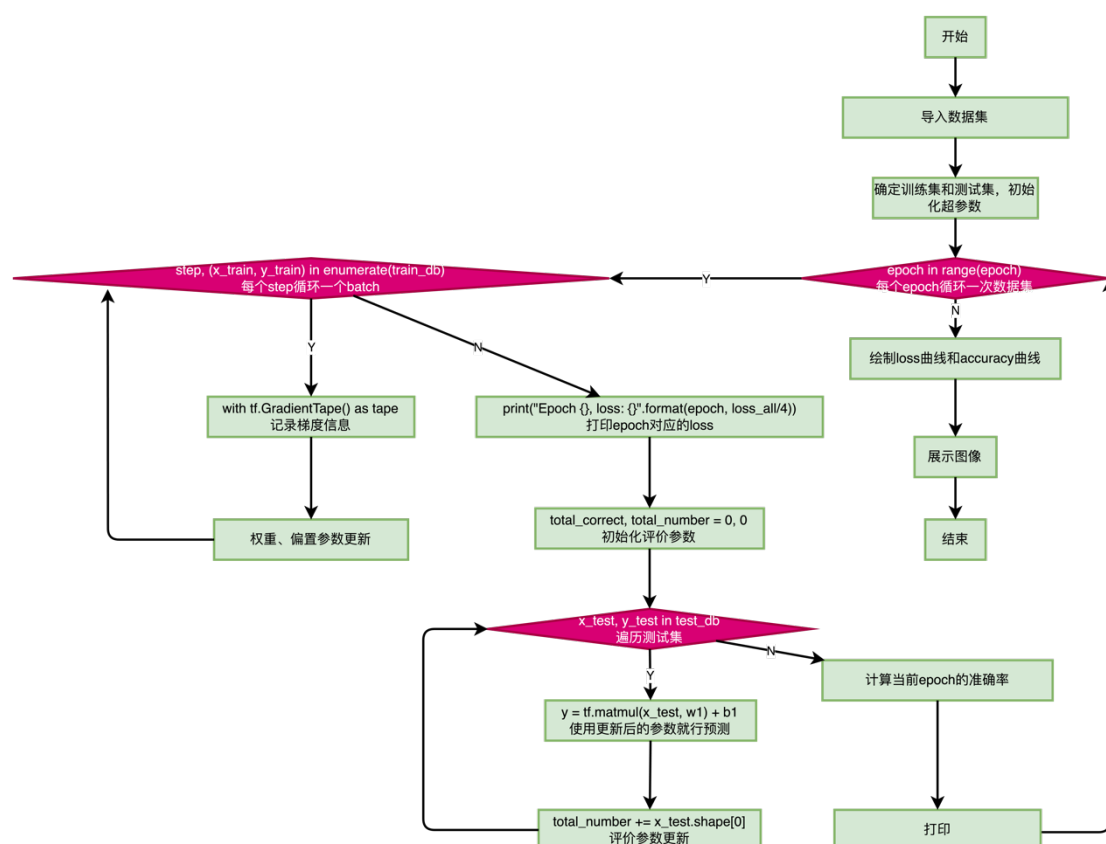


图 4 鸢尾花训练程序流程图

以上流程图的进一步文字说明如下：

1. 开始：流程的起始点。
2. 导入所需模块：导入 TensorFlow、Sklearn 中的 datasets、Matplotlib 的 pyplot 以及 NumPy。
3. 载入数据集：加载鸢尾花数据集的特征和标签。
4. 数据打乱：为了确保模型训练的泛化能力，通过随机数种子将数据顺序打乱。
5. 划分数据集：将打乱的数据分为训练集和测试集，通常是按照一定比例分配。
6. 数据类型转换：将数据集的类型转换为适合模型输入的格式。
7. 创建数据批次：将数据分批处理，便于后续的批量训练。
8. 初始化参数：初始化模型的权重（ w_1 ）和偏置（ b_1 ）参数。
9. 设置学习参数：定义学习率等参数，设置训练的轮数（epoch）。
10. epoch 循环：针对每个 epoch，循环进行训练和测试。
 - ✧ 前向传播：计算模型的输出。
 - ✧ 计算 loss：根据模型输出和真实标签计算损失函数。
 - ✧ 反向传播：基于损失函数计算参数的梯度。
 - ✧ 更新参数：根据梯度和学习率更新模型参数。
 - ✧ 计算准确率：在测试数据集上计算模型的准确率。
11. 绘制并保存曲线：将训练过程中的 loss 和 accuracy 变化曲线绘制出来，并保存为图片文件。
12. 结束：流程的终点，得到模型训练和测试的结果。

5.3 单层感知机神经网络结构

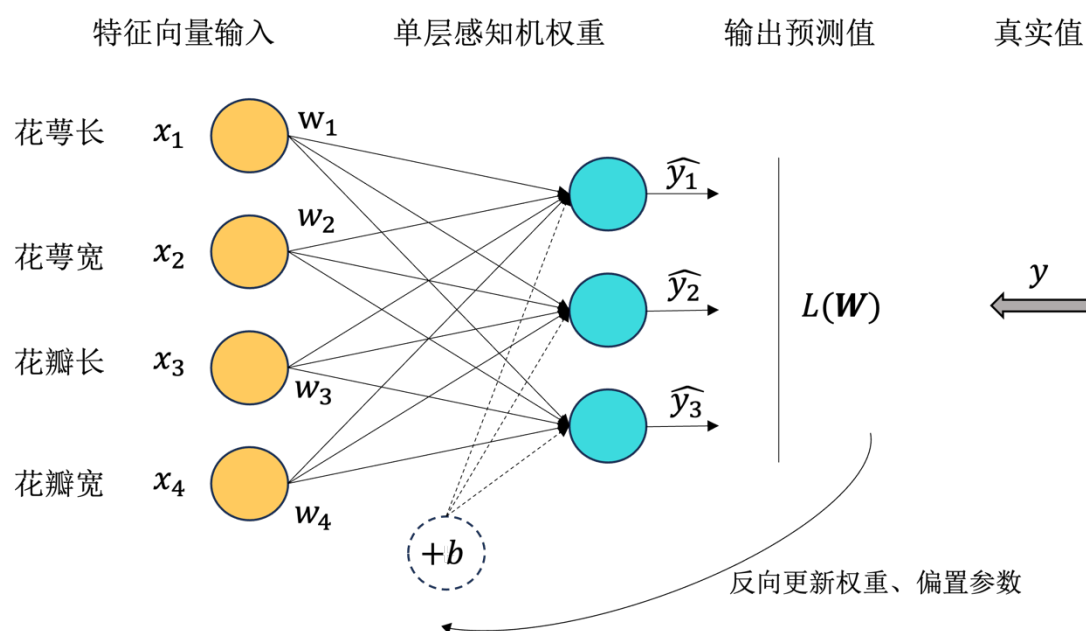


图 5 单层感知机神经网络结构图

由上图，单层感知机神经网络结构主要由以下几个部分组成：

1. 输入层 (Input Layer):

这一层由数据集的特征决定，在鸢尾花数据集中有 4 个特征：花萼长度、花萼宽度、花瓣长度和花瓣宽度。因此，输入层有 4 个神经元，每个神经元对应于一个特征。

2. 输出层 (Output Layer):

鉴于鸢尾花数据集是一个三分类问题 (Setosa, Versicolour, 和 Virginica)，输出层有 3 个神经元。每个神经元代表一个类别的得分。最高得分的神经元对应的类别即为模型的预测结果。

3. 权重和偏置 (Weights and Biases):

每个输入特征都通过一系列权重与输出层相连接，权重表示输入特征与结果之间的关系强度和方向。

每个输出神经元还有一个偏置，能够调整输出神经元的激活阈值。

4. 激活函数 (Activation Function):

在单层感知机中，通常不会对输入层应用激活函数，而是直接将加权和传递到输出层。

输出层通常会使用激活函数来转换神经元的加权和。在多类分类问题中，

softmax 函数被用来将输出转换为概率分布。

5. 损失函数 (Loss Function):

为了训练网络，需要一个损失函数来衡量预测值和真实值之间的差异。在这段代码中使用了均方误差 (Mean Squared Error, MSE)，它量化了预测误差的平方的平均值。

6. 优化和反向传播 (Optimization and Backpropagation):

通过计算损失函数相对于权重的梯度，并使用这些梯度来更新权重，这个过程称为反向传播。梯度告诉我们如何改变权重以减少损失。

学习率 (Learning Rate) 决定了在每次迭代中更新权重的幅度。太大可能会导致超调，太小则可能会导致收敛速度过慢。

六、成绩